Тема 15: Структури от данни. Стек, опашка, списък, дърво.

⚠ На изпита се избират **2 от 6** структури. Всички са описани тук.

---

0. Дефиниция на структура от данни

Структура от данни = схема за организация на данни в паметта, обикновено с цел ефективност на операциите.

Критерий	**Видове**
Компоненти	Хомогенни (масив) / Хетерогенни (struct)
Размер	Статични / Динамични
Връзки	Линейни (списък, стек) / Разклонени (дърво) / Мрежови (граф)

---

1. СВЪРЗАН СПИСЪК (Linked List)

Логическо описание: Хомогенна линейна структура с **последователен достъп**. Елементите са свързани чрез указатели. Без предварително зададен размер.

Операции:

- `empty()` — проверка за празен
- `push_front(x)` / `push_back(x)` — добавяне
- `pop_front()` / `pop_back()` — премахване
- `insert(x, pos)` / `delete(pos)` — по позиция
- `find(x)` — търсене

1.1 Сингълно-свързан списък (SLL)

```
[data|next] → [data|next] → [data|next] → nullptr
   head                          tail

template <typename T>
class SLinkedList {
    struct Node {
        T data;
        Node* next;
        Node(const T& d) : data(d), next(nullptr) {}
    };
    Node* head;
    Node* tail;
    size_t sz;
```

```
public:
    SLinkedList() : head(nullptr), tail(nullptr), sz(0) {}

    // Copy constructor
    SLinkedList(const SLinkedList& other) : head(nullptr), tail(nullptr), sz(0)
{
        Node* cur = other.head;
        while (cur) { push_back(cur->data); cur = cur->next; }
    }

    SLinkedList& operator=(const SLinkedList& other) {
        if (this != &other) { clear(); SLinkedList tmp(other); swap(tmp); }
        return *this;
    }

    ~SLinkedList() { clear(); }

    void push_front(const T& val) {           // O(1)
        Node* n = new Node(val);
        n->next = head;
        head = n;
        if (!tail) tail = head;
        sz++;
    }

    void push_back(const T& val) {            // O(1) – маинтейнваме tail
        Node* n = new Node(val);
        if (!tail) { head = tail = n; }
        else { tail->next = n; tail = n; }
        sz++;
    }

    void pop_front() {                        // O(1)
        if (!head) return;
        Node* old = head;
        head = head->next;
        if (!head) tail = nullptr;
        delete old; sz--;
    }

    void pop_back() {                         // O(n) – трябва да намерим
предпоследния
        if (!head) return;
        if (head == tail) { delete head; head = tail = nullptr; sz--; return; }
        Node* cur = head;
        while (cur->next != tail) cur = cur->next;
        delete tail;
        tail = cur;
        tail->next = nullptr;
        sz--;
    }

    T& front() const { return head->data; }
    T& back()  const { return tail->data; }
    bool empty() const { return sz == 0; }
    size_t size() const { return sz; }

    void clear() {
        while (head) pop_front();
    }

private:
    void swap(SLinkedList& other) {
        std::swap(head, other.head);
```

```
        std::swap(tail, other.tail);
        std::swap(sz, other.sz);
    }
};
```

1.2 Двойно-свързан списък (DLL)

```
nullptr ← [prev|data|next] ↔ [prev|data|next] ↔ [prev|data|next] → nullptr
           head                                        tail
```

Разлика от SLL: всеки Node има `prev` указател.

```
struct Node {
    T data;
    Node* next;
    Node* prev;
};
```

Предимство: `pop_back()` е O(1) (без нужда да обхождаме).

Сложности:

Операция	**SLL**	**DLL**
`push_front`	O(1)	O(1)
`push_back`	O(1)	O(1)
`pop_front`	O(1)	O(1)
`pop_back`	**O(n)**	**O(1)**
`insert(pos)`	O(n)	O(n)
`delete(pos)`	O(n)	O(n)
`find(x)`	O(n)	O(n)

---

2. СТЕК (Stack)

Логическо описание: Хомогенна линейна структура **LIFO** (Last In, First Out).
Достъп само до **върха** (top).

Операции:

- `push(x)` — добавяне на върха
- `pop()` — премахване от върха
- `peek()/top()` — поглед на върха без премахване
- `empty()` — проверка

2.1 Статична реализация (масив с фиксиран размер)

```
template <typename T>
class StackStatic {
```

```
    T    data[MAX_SIZE];
    int  topIdx;
public:
    StackStatic() : topIdx(-1) {}
    void push(const T& val) { data[++topIdx] = val; }  // O(1)
    void pop()              { topIdx--; }              // O(1)
    T&   top()              { return data[topIdx]; }   // O(1)
    bool empty() const      { return topIdx == -1; }
    bool full()  const      { return topIdx == MAX_SIZE - 1; }
};
```

Предимства: много бърз, без алокация. **Недостатъци**: фиксиран размер, препълване.

2.2 Динамична реализация (динамичен масив)

```
template <typename T>
class StackDynamic {
    T*     data;
    int    topIdx;
    int    capacity;

    void resize() {
        capacity *= 2;
        T* newData = new T[capacity];
        for (int i = 0; i <= topIdx; i++) newData[i] = data[i];
        delete[] data;
        data = newData;
    }
public:
    StackDynamic(int cap = 8) : topIdx(-1), capacity(cap) { data = new T[cap]; }
    ~StackDynamic() { delete[] data; }

    void push(const T& val) {                          // O(1) амортизирано
        if (topIdx + 1 == capacity) resize();
        data[++topIdx] = val;
    }
    void pop()  { topIdx--; }                          // O(1)
    T&   top()  { return data[topIdx]; }               // O(1)
    bool empty() const { return topIdx == -1; }
};
```

Предимства: гъвкав размер. **Недостатъци**: resize е O(n) (рядко).

2.3 Свързана реализация (чрез SLL)

```
template <typename T>
class StackLinked {
    struct Node { T data; Node* next; };
    Node* topNode;
public:
    StackLinked() : topNode(nullptr) {}
    ~StackLinked() { while (!empty()) pop(); }

    void push(const T& val) {                          // O(1)
        Node* n = new Node{val, topNode};
        topNode = n;
    }
    void pop() {                                       // O(1)
        Node* old = topNode;
        topNode = topNode->next;
        delete old;
    }
```

```
    T&   top()   { return topNode->data; }              // O(1)
    bool empty() const { return topNode == nullptr; }
};
```

Предимства: неограничен размер, O(1) push/pop. **Недостатъци**: overhead от указатели.

Сравнение реализации на стек:

Реализация	**push**	**pop**	**Памет**	**Ограничение**
Статична	O(1)	O(1)	фиксирана	препълване
Динамична	O(1)*	O(1)	гъвкава	realloc
Свързана	O(1)	O(1)	гъвкава	pointer overhead

---

3. ОПАШКА (Queue)

Логическо описание: Хомогенна линейна структура **FIFO** (First In, First Out).
Добавяне на **опашката**, премахване от **началото**.

Операции:

- `enqueue(x)` — добавяне в края
- `dequeue()` — премахване от началото
- `front()` — поглед на началото
- `empty()`

3.1 Статична циркулярна реализация (Ring Buffer)

```
template <typename T>
class QueueCircular {
    T   data[MAX_SIZE];
    int frontIdx, backIdx, sz;
public:
    QueueCircular() : frontIdx(0), backIdx(0), sz(0) {}

    void enqueue(const T& val) {                        // O(1)
        data[backIdx] = val;
        backIdx = (backIdx + 1) % MAX_SIZE;
        sz++;
    }
    void dequeue() {                                    // O(1)
        frontIdx = (frontIdx + 1) % MAX_SIZE;
        sz--;
    }
    T&   front() { return data[frontIdx]; }             // O(1)
    bool empty() const { return sz == 0; }
    bool full()  const { return sz == MAX_SIZE; }
};
```

Защо циркулярна: без circular buffer при обикновен масив frontIdx нараства и "губим" памет.

3.2 Динамична реализация (два указателя)

Аналогична на статичната, но с resize при препълване.

3.3 Свързана реализация (чрез SLL)

```
template <typename T>
class QueueLinked {
    struct Node { T data; Node* next; };
    Node* frontNode;
    Node* backNode;
public:
    QueueLinked() : frontNode(nullptr), backNode(nullptr) {}
    ~QueueLinked() { while (!empty()) dequeue(); }

    void enqueue(const T& val) {                        // O(1)
        Node* n = new Node{val, nullptr};
        if (!backNode) { frontNode = backNode = n; }
        else { backNode->next = n; backNode = n; }
    }
    void dequeue() {                                    // O(1)
        Node* old = frontNode;
        frontNode = frontNode->next;
        if (!frontNode) backNode = nullptr;
        delete old;
    }
    T&   front() { return frontNode->data; }            // O(1)
    bool empty() const { return frontNode == nullptr; }
};
```

3.4 Опашка чрез два стека

```
template <typename T>
class QueueViaStacks {
    StackLinked<T> inbox;   // enqueue тук
    StackLinked<T> outbox;  // dequeue оттук

    void transfer() {  // пренасяме inbox → outbox (обръща реда)
        while (!inbox.empty()) {
            outbox.push(inbox.top());
            inbox.pop();
        }
    }
public:
    void enqueue(const T& val) { inbox.push(val); }     // O(1)
    void dequeue() {                                    // O(1) амортиз.
        if (outbox.empty()) transfer();
        outbox.pop();
    }
    T& front() {
        if (outbox.empty()) transfer();
        return outbox.top();
    }
    bool empty() const { return inbox.empty() && outbox.empty(); }
};
```

Сравнение реализации на опашка:

Реализация	**enqueue**	**dequeue**	**Бележка**
Циркулярна статична	O(1)	O(1)	фиксиран размер

Реализация	enqueue	dequeue	Бележка
Свързана	O(1)	O(1)	гъвкава
Два стека	O(1)	O(1)*	амортизирано

---

4. ДВОИЧНО КОРЕНОВО ДЪРВО (Binary Tree)

Логическо описание: Всеки възел има **най-много две деца** — ляво и дясно.

```
        1
      /   \
     2     3
    / \     \
   4   5     6
```

Терминология: корен, лист, родител, дете, ниво, височина, ширина.

Обхождания:

- Префиксно (Preorder): **Корен** → Ляво → Дясно (КЛД)
- Инфиксно (Inorder): Ляво → **Корен** → Дясно (ЛКД) ← за BST дава сортирана редица!
- Постфиксно (Postorder): Ляво → Дясно → **Корен** (ЛДК)
- По нива (BFS): ред по ред отляво надясно

Свързано представяне (указатели)

```
template <typename T>
struct BTreeNode {
    T data;
    BTreeNode* left;
    BTreeNode* right;
    BTreeNode(const T& d) : data(d), left(nullptr), right(nullptr) {}
};

template <typename T>
class BinaryTree {
    BTreeNode<T>* root;

    void printInorder(BTreeNode<T>* node) const {
        if (!node) return;
        printInorder(node->left);
        cout << node->data << " ";
        printInorder(node->right);
    }

    int height(BTreeNode<T>* node) const {
        if (!node) return 0;
        return 1 + max(height(node->left), height(node->right));
    }

    BTreeNode<T>* copyTree(BTreeNode<T>* node) {
        if (!node) return nullptr;
        BTreeNode<T>* n = new BTreeNode<T>(node->data);
        n->left  = copyTree(node->left);
        n->right = copyTree(node->right);
```

```
        return n;
    }

    void freeTree(BTreeNode<T>* node) {
        if (!node) return;
        freeTree(node->left);
        freeTree(node->right);
        delete node;
    }

public:
    BinaryTree() : root(nullptr) {}
    BinaryTree(const BinaryTree& other) : root(copyTree(other.root)) {}
    ~BinaryTree() { freeTree(root); }

    void inorder()  const { printInorder(root); cout << endl; }
    int  height()   const { return height(root); }
};
```

Последователно представяне (масив)

За **пълно двоично дърво** с корен на индекс 1:

- Ляво дете на i → `2*i`
- Дясно дете на i → `2*i + 1`
- Родител на i → `i/2`

```
T tree[MAX]; // 1-базиран масив
```

Предимство: без указатели, cache-friendly. **Недостатък**: за неплъно дърво — много дупки.

---

5. ДВОИЧНО ДЪРВО ЗА ТЪРСЕНЕ (BST — Binary Search Tree)

BST свойство: за всеки възел v:

- Всички стойности в **лявото** поддърво < v
- Всички стойности в **дясното** поддърво > v

Инфиксно обхождане на BST → сортирана редица!

```
template <typename T>
class BST {
    struct Node {
        T data;
        Node* left;
        Node* right;
        Node(const T& d) : data(d), left(nullptr), right(nullptr) {}
    };
    Node* root;

    // Помощни рекурсивни функции
    Node* insert(Node* node, const T& val) {
        if (!node) return new Node(val);
        if (val < node->data) node->left  = insert(node->left, val);
        else if (val > node->data) node->right = insert(node->right, val);
```

```
        // val == node->data: игнорираме (без дублирания)
        return node;
    }

    bool search(Node* node, const T& val) const {
        if (!node) return false;
        if (val == node->data) return true;
        if (val < node->data) return search(node->left, val);
        return search(node->right, val);
    }

    Node* findMin(Node* node) const {
        while (node->left) node = node->left;
        return node;
    }

    Node* remove(Node* node, const T& val) {
        if (!node) return nullptr;
        if (val < node->data) {
            node->left = remove(node->left, val);
        } else if (val > node->data) {
            node->right = remove(node->right, val);
        } else {
            // Намерен!
            if (!node->left) {        // 0 или 1 дете (дясно)
                Node* r = node->right;
                delete node; return r;
            }
            if (!node->right) {       // 1 дете (ляво)
                Node* l = node->left;
                delete node; return l;
            }
            // 2 деца: заменяме с in-order successor (мин. от дясното поддърво)
            Node* successor = findMin(node->right);
            node->data = successor->data;
            node->right = remove(node->right, successor->data);
        }
        return node;
    }

    void inorder(Node* node) const {
        if (!node) return;
        inorder(node->left);
        cout << node->data << " ";
        inorder(node->right);
    }

    void free(Node* node) {
        if (!node) return;
        free(node->left);
        free(node->right);
        delete node;
    }

public:
    BST() : root(nullptr) {}
    ~BST() { free(root); }

    void insert(const T& val) { root = insert(root, val); }
    bool search(const T& val) const { return search(root, val); }
    void remove(const T& val) { root = remove(root, val); }
    void print() const { inorder(root); cout << endl; }
};
```

Случаи при `remove`:

1. **Листо**: просто изтрий
2. **1 дете**: свържи родителя директно с детето
3. **2 деца**: намери in-order successor (минималния от дясното поддърво), копирай стойността, изтрий successor

Сложност на BST:

Операция	Средно (балансирано)	Най-лошо (дегенерирало)
`insert`	O(log n)	O(n)
`search`	O(log n)	O(n)
`remove`	O(log n)	O(n)

Дегенерирало дърво = вмъкване в сортиран ред → списък.

Балансирани BST: AVL, Червено-черно, 2-3 дърво → гарантирано O(log n).

КАК СЕ РЕШАВАТ ЗАДАЧИ

Task 1 или Task 2 на практическия изпит — код с СДД.

Тип 1: Имплементирай структура с дадена реализация

Стратегия: Знай структурата на Node + Rule of Three за клас с динамична памет.

Чеклист за всяка СДД:

- ☐ Дефиниция на Node struct
- ☐ Конструктор (default)
- ☐ Copy constructor + operator= (deep copy)
- ☐ Destructor (освобождава всички Node-ове)
- ☐ Основните операции с правилна сложност

Тип 2: Trace на операции върху BST

Вмъкни 5, 3, 7, 1, 4:

```
    5
   / \
  3   7
 / \
1   4
```

Inorder: 1 3 4 5 7 (сортирано ✓)

Изтрий 3 (2 деца → in-order successor = 4):

```
    5
   / \
  4   7
 /
1
```

Тип 3: Опашка чрез стек / Стек чрез опашка

Стек чрез 2 опашки:

```
// push: enqueue в q1
// pop: прехвърли всичко от q1 в q2 без последния, върни последния
```

Бърза справка

Структура	**Основна идея**	**push**	**pop**
Стек (LIFO)	last in, first out	O(1)	O(1)
Опашка (FIFO)	first in, first out	O(1)	O(1)
SLL	указатели; pop_back = O(n)	O(1)	O(1)/O(n)
DLL	двойни указатели	O(1)	O(1)
BST	ляво<node<дясно	O(log n)	O(log n)
Inorder BST	дава сортирана редица	—	—
Remove 2 деца	замени с in-order successor	—	—